Build a Real-Time Cyber Threat Detection Engine Using Hashing and Self-Balancing Trees

ASSIGNMENT REPORT

Submitted in the partial fulfilment for the Course

of

CSA0309 - Data Structures

to the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE ENGINEERING

B Sathvika (192525224)
PRENA M (192524056)

Submitted by

Under the Supervision of

Dr. Uma Priyadarsini P.S

Saveetha School of Engineering

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

September 2026

COURSE ASSIGNMENT

A. Assignment Information

Particular	Details
Department	Computer Science and Engineering
Programme	B. Tech – Artificial Intelligence & Data Science
Course Code & Course Name	CSA0309 – Data Structures
Academic Year / Batch	2026 – 2027
Faculty Name	Dr Uma Priyadarsini P S
Assignment Title	Build a Real-Time Cyber Threat Detection Engine Using Hashing and Self-Balancing Trees
Date of Issue	31/08/2026
Date of Submission	02/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO2: Apply the suitable Abstract Data Type for construction of the high-speed security log processing system. CO3: Make use of self-balancing trees (AVL/Red-Black Trees) for ordered, dynamically balanced storage of risk-ranked events. CO4: Make use of hashing, collision-resolution techniques, and priority queues for duplicate detection and high-risk event ranking in C. CO5: Develop a robust real-time cyber threat detection engine and evaluate its search performance under varying load factors and data distributions.
Bloom's Taxonomy Level	L4/L5 – Analyze / Evaluate / Create
SDG Mapping	SDG 9 – Industry, Innovation and Infrastructure SDG 16 – Peace, Justice and Strong Institutions
Industry / Societal Relevance	The project addresses real-time cybersecurity monitoring by efficiently processing large volumes of security events, identifying repeated or suspicious IP addresses, ranking high-risk events, and supporting rapid threat detection.

B. Assignment Problem / Challenge

The assignment focuses on developing a real-time cyber threat detection engine capable of processing a large volume of security event logs efficiently. The system must use suitable data structures and algorithms to identify duplicate IP addresses, store events according to their risk levels, and continuously identify the most critical threats.

The system should demonstrate the practical application of hashing, collision-resolution techniques, self-balancing binary search trees, and priority queues

The assignment also requires performance evaluation of the hash table under different load factors and different key/data distributions. The results should be analyzed to determine how the selected data structures perform when the number of events and distinct IP addresses becomes very large.

C. Problem Statement

A cybersecurity operations center receives a continuous stream of security event logs such as login attempts, IP connections, and access requests. Each event contains information such as source IP address, timestamp, event type, and risk indicators.

Since the number of events may exceed 1,000,000, the system requires efficient data structures for fast insertion, searching, duplicate detection, and risk ranking.

The proposed cyber threat detection engine must:

1. Process a high-volume stream of security events.
2. Detect duplicate or repeated IP addresses quickly.
3. Identify suspicious activity such as repeated login attempts and scanning patterns.
4. Store and retrieve events efficiently.
5. Maintain events or IP addresses in risk-score order.
6. Continuously identify and display the highest-risk events.
7. Handle collisions in the hash table efficiently.
8. Remain efficient under different load factors.
9. Handle skewed data distributions where a small number of malicious IP addresses generate a large percentage of events.
10. Compare the performance of different hashing and collision-resolution strategies.

The system should use a hash table for average O(1) duplicate-IP and event lookup, an AVL Tree or Red-Black Tree for ordered risk-based storage, and a priority queue for continuously identifying the highest-risk events.

D. Requirements and Constraints

Functional Requirements

1. Accept security events containing:
    - Source IP address
    - Timestamp
    - Event type
    - Risk score or risk indicators
2. Insert each event into the hash table.
3. Search for existing IP addresses using hashing.
4. Detect repeated IP addresses.
5. Maintain risk-ranked events using an AVL Tree or Red-Black Tree.
6. Insert high-risk events into a priority queue.
7. Retrieve the highest-risk event efficiently.
8. Generate ordered reports of suspicious events.
9. Perform range queries based on risk scores.
10. Record performance measurements for insertion and search operations.

Performance Requirements

The system should provide:

- O(1) average-case hash table insertion and lookup.
- O(log n) operations for the self-balancing tree.
- Efficient highest-risk event retrieval using a priority queue.
- Minimal latency for high-volume event processing.
- Stable performance when the load factor changes.

Technical Constraints

- Programming language: C
- Data structures: Hash Table, AVL Tree or Red-Black Tree, Priority Queue
- Collision handling: Chaining or Open Addressing
- Dataset: Large security-event dataset or generated test data
- Performance measurement: Execution time and operation counts
- Version control: GitHub

Reliability and Security Requirements

The system should correctly handle:

- Duplicate IP addresses
- Hash collisions
- Large numbers of events
- Repeated malicious IP addresses
- Empty or invalid inputs
- Different risk levels
- Highly skewed event distributions

Sustainability and Societal Considerations

Efficient data structures reduce computational overhead and enable security monitoring systems to process large amounts of data without unnecessary resource consumption. The system also supports improved cybersecurity infrastructure and stronger protection of digital services.

E. Student Work

1. Problem Understanding and Formulation

The main problem is to design a high-speed cyber threat detection engine that can process a very large number of security events while maintaining low search and insertion latency.

A conventional linear search would become inefficient when the number of events reaches millions. Therefore, hashing is used to provide average O(1) lookup for IP addresses. However, collisions can occur when multiple IP addresses map to the same hash-table index, so an appropriate collision-resolution technique is required.

A self-balancing tree is used when ordered storage and range-based searching are required. Unlike a hash table, a balanced tree maintains elements in sorted order and supports operations in O(log n) time.

A priority queue is used to continuously retrieve the event having the highest risk score. This allows the security operations center to focus on the most critical threats first.

Expected Outcomes

The proposed system should:

- Detect duplicate IP addresses efficiently.
- Process a large number of events.
- Maintain risk-ranked events.
- Identify the highest-risk events quickly.
- Support ordered reporting and range queries.
- Demonstrate the effect of different load factors on hash-table performance.
- Demonstrate the effect of different key distributions on performance.
- Provide experimental results that justify the selected data structures.

Assumptions

1. Each security event has a valid source IP address.
2. Each event is assigned a risk score.
3. The risk score is represented using a numerical value.
4. The hash table has a configurable capacity.
5. The test dataset may be generated programmatically to simulate millions of events.
6. The priority queue treats a higher risk score as a higher priority.

Constraints

The major constraints are the large number of events, duplicate IP addresses, hash collisions, changing load factors, and skewed data distributions. The implementation must also manage memory carefully because millions of events can require significant storage.

2. Application of Course Knowledge

The following data structures are applied to solve the problem:

Hash Table

The hash table stores IP addresses and associated event information. A hash function converts an IP address into an index.

Average-case complexity:

- Search: O(1)
- Insertion: O(1)
- Deletion: O(1)

Worst-case complexity can become O(n) when many keys collide.

Collision Resolution

Two possible collision-resolution techniques are:

1. Separate Chaining
2. Open Addressing

Separate chaining stores multiple entries belonging to the same hash index using a linked structure. Open addressing searches for another available position within the hash table.

For this implementation, separate chaining can be selected because it handles duplicate and skewed IP distributions more flexibly and does not require finding an unused table slot for every collision.

AVL Tree

The AVL Tree maintains events according to their risk score while automatically maintaining balance.

For n elements:

- Search: O(log n)
- Insertion: O(log n)
- Deletion: O(log n)

The AVL Tree is useful for ordered reporting and range queries.

Priority Queue

The priority queue stores events according to their risk scores. The event with the highest risk is placed at the highest priority.

Using a binary max-heap:

- Insertion: O(log n)
- Highest-risk event retrieval: O(1)
- Deletion of highest-risk event: O(log n)

Overall Data-Structure Design

The system combines the three structures as follows:

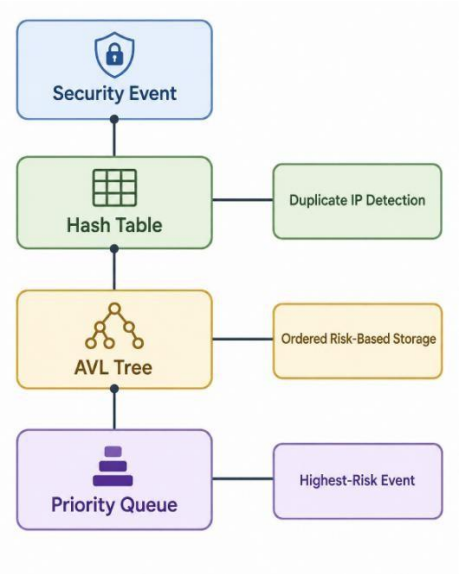

Figure 1: Data Structure Design

This combination allows the system to perform fast duplicate detection while also supporting ordered searches and immediate access to critical events.

3. Solution / Design / Methodology

Step 1: Read Security Event

The system receives the source IP, timestamp, event type, and risk score.

Step 2: Calculate Hash Value

The source IP address is passed through a hash function to calculate the corresponding hash-table index.

Step 3: Search Hash Table

The system checks whether the IP address already exists.

If the IP exists, its event count is increased and the repeated activity can be treated as a possible threat.

If the IP does not exist, a new entry is inserted.

Step 4: Insert into AVL Tree

The event is inserted into the AVL Tree according to its risk score. The tree performs rotations whenever required to maintain balance.

Step 5: Insert into Priority Queue

The event is inserted into a max-priority queue so that the event with the highest risk score can be retrieved quickly.

Step 6: Detect High-Risk Events

The system examines the highest-priority event and displays it to the security operator.

Step 7: Generate Reports

The AVL Tree can be traversed to generate a sorted list of events according to risk score.

Step 8: Performance Testing

The hash table is tested using different load factors and different distributions of IP addresses.

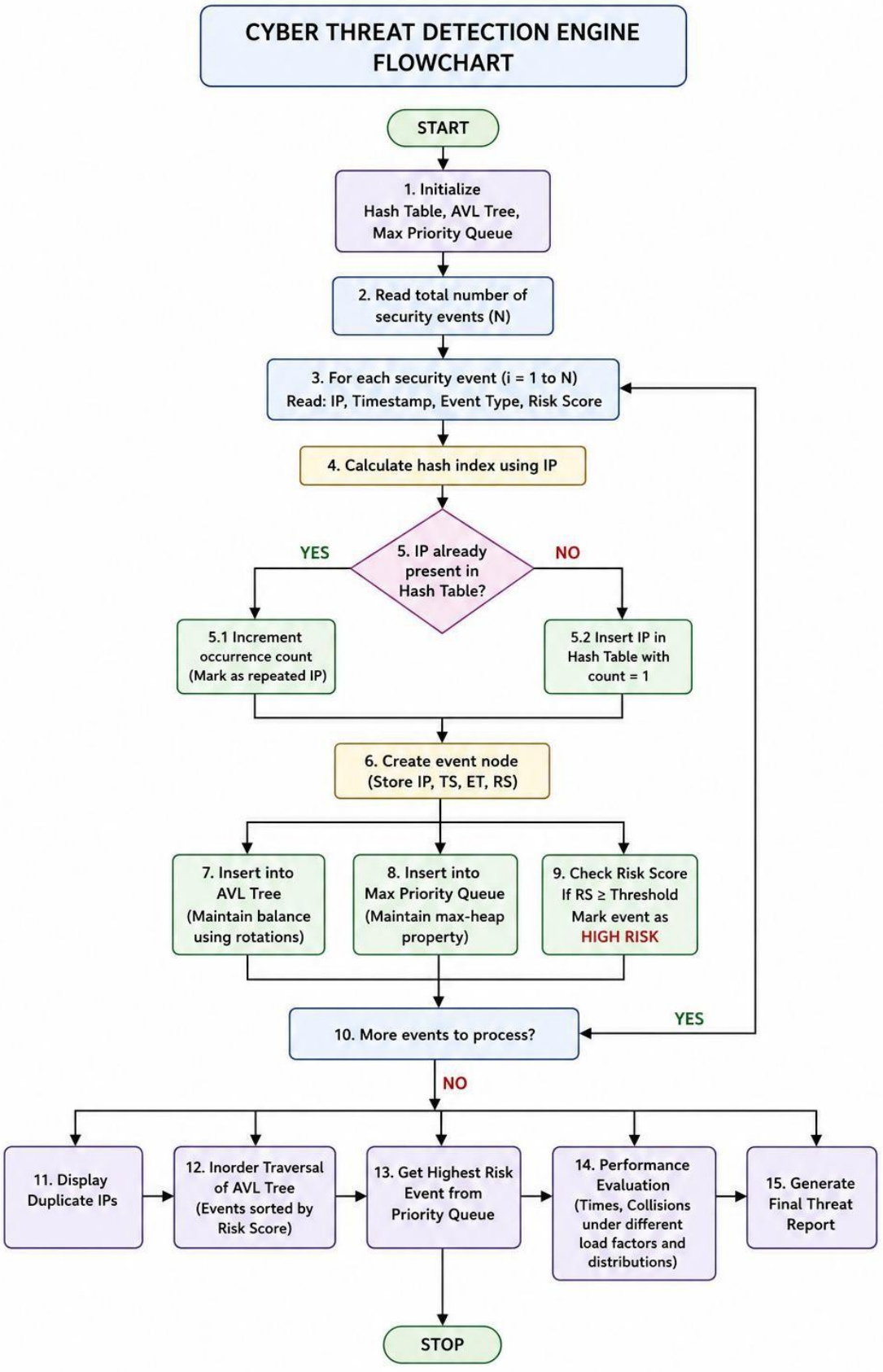

Figure 2: Flowchart

Proposed Pseudocode

```
START
Initialize hash table
Initialize AVL Tree
Initialize priority queue
FOR each incoming security event
Read source IP
Read timestamp
Read event type
Read risk score
index = Hash(source IP)
Search source IP in hash table
IF IP exists THEN
   Increase IP event count
   Mark repeated activity
ELSE
   Insert new IP into hash table
END IF
Insert event into AVL Tree using risk score
Insert event into priority queue
IF priority queue is not empty THEN
   Display highest-risk event
END IF
END FOR
Generate ordered risk report
Measure search and insertion performance
Display performance results
STOP
```

Hash Table Pseudocode

```
HASH_SEARCH(IP)
index = HASH(IP)
Traverse the chain at index
IF matching IP is found THEN
   RETURN IP entry
ELSE
   RETURN NOT_FOUND
END IF
END
```

AVL Tree Insertion Pseudocode

```
AVL_INSERT(root, event)
IF root is NULL THEN
   Create new node
   RETURN node
END IF
IF event risk score < root risk score THEN
   root.left = AVL_INSERT(root.left, event)
```

```
ELSE
   root.right = AVL_INSERT(root.right, event)
END IF
Update height of root
Calculate balance factor
IF tree becomes unbalanced THEN
   Perform appropriate AVL rotation
END IF
RETURN root
END
```

Priority Queue Pseudocode

```
INSERT_PRIORITY(event)
Insert event at the end of the heap
Compare event with its parent
WHILE event has higher priority than parent
   Swap event and parent
END WHILE
END
EXTRACT_HIGH_RISK()
IF queue is empty THEN
   RETURN EMPTY
END IF
RETURN stored high-risk event
END
```

Implementation

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#define TABLE_SIZE 1009
#define MAX_EVENTS 10000
typedef struct {
   char ip[16];
   char timestamp[30];
   char eventType[30];
   int riskScore;
} Event;
typedef struct HashNode {
   char ip[16];
   int count;
   struct HashNode *next;
} HashNode;
HashNode *hashTable[TABLE_SIZE];
unsigned long hashFunction(char *ip) {
   unsigned long hash = 5381;
   int c;
```

```
    while ((c = *ip++))
        hash = ((hash << 5) + hash) + c;
    return hash % TABLE_SIZE;
}
void insertHash(char *ip) {
    unsigned long index = hashFunction(ip);
    HashNode *current = hashTable[index];
    while (current != NULL) {
        if (strcmp(current->ip, ip) == 0) {
            current->count++;
            return;
        }
        current = current->next;
    }
    HashNode *newNode =
        (HashNode *)malloc(sizeof(HashNode));
    strcpy(newNode->ip, ip);
    newNode->count = 1;
    newNode->next = hashTable[index];
    hashTable[index] = newNode;
}
int searchHash(char *ip) {
    unsigned long index = hashFunction(ip);
    HashNode *current = hashTable[index];
    while (current != NULL) {
        if (strcmp(current->ip, ip) == 0)
            return current->count;
        current = current->next;
    }
    return 0;
}
typedef struct AVLNode {
    Event;
    int height;
    struct AVLNode *left;
    struct AVLNode *right;
} AVLNode;
int height(AVLNode *node) {
    if (node == NULL)
        return 0;
    return node->height;
}
int max(int a, int b) {
    return (a > b) ? a : b;
}
AVLNode *createAVLNode(Event event) {
```

```
    AVLNode *node =
        (AVLNode *)malloc(sizeof(AVLNode));
    node->event = event;
    node->height = 1;
    node->left = NULL;
    node->right = NULL;
    return node;
}
AVLNode *rightRotate(AVLNode *y) {
    AVLNode *x = y->left;
    AVLNode *T2 = x->right;
    x->right = y;
    y->left = T2;
    y->height =
        max(height(y->left), height(y->right)) + 1;
    x->height =
        max(height(x->left), height(x->right)) + 1;
    return x;
}
AVLNode *leftRotate(AVLNode *x) {
    AVLNode *y = x->right;
    AVLNode *T2 = y->left;
    y->left = x;
    x->right = T2;
    x->height =
        max(height(x->left), height(x->right)) + 1;
    y->height =
        max(height(y->left), height(y->right)) + 1;
    return y;
}
int getBalance(AVLNode *node) {
    if (node == NULL)
        return 0;
    return height(node->left) -
           height(node->right);
}
AVLNode *insertAVL(AVLNode *root, Event event) {
    if (root == NULL)
        return createAVLNode(event);
    if (event.riskScore < root->event.riskScore)
        root->left =
            insertAVL(root->left, event);
    else
        root->right =
            insertAVL(root->right, event);
    root->height =
        1 + max(height(root->left),
```

```
            height(root->right));
    int balance = getBalance(root);
    /* Left Case */
    if (balance > 1 &&
        event.riskScore < root->left->event.riskScore)
        return rightRotate(root);
    /* Right Case */
    if (balance < -1 &&
        event.riskScore >= root->right->event.riskScore)
        return leftRotate(root);
    /* Left Right Case */
    if (balance > 1 &&
        event.riskScore >= root->left->event.riskScore) {
        root->left = leftRotate(root->left);
        return rightRotate(root);
    }
    /* Right Left Case */
    if (balance < -1 &&
        event.riskScore < root->right->event.riskScore) {
        root->right = rightRotate(root->right);
        return leftRotate(root);
    }
    return root;
}
void inorder(AVLNode *root) {
    if (root != NULL) {
        inorder(root->left);
        printf("IP: %-15s Risk: %d Type: %s\n",
               root->event.ip,
               root->event.riskScore,
               root->event.eventType);
        inorder(root->right);
    }
}
Event priorityQueue[MAX_EVENTS];
int pqSize = 0;
void swapEvent(Event *a, Event *b) {
    Event temp = *a;
    *a = *b;
    *b = temp;
}
void insertPriority(Event event) {
    int i = pqSize++;
    priorityQueue[i] = event;
    while (i != 0) {
        int parent = (i - 1) / 2;
        if (priorityQueue[parent].riskScore >=
```

```
            priorityQueue[i].riskScore)
            break;
        swapEvent(&priorityQueue[parent],
                &priorityQueue[i]);
        i = parent;
    }
}
Event getHighestRisk() {
    Event result = priorityQueue[0];
    priorityQueue[0] =
        priorityQueue[--pqSize];
    int i = 0;
    while (1) {
        int left = 2 * i + 1;
        int right = 2 * i + 2;
        int largest = i;
        if (left < pqSize &&
            priorityQueue[left].riskScore >
            priorityQueue[largest].riskScore)
            largest = left;
        if (right < pqSize &&
            priorityQueue[right].riskScore >
            priorityQueue[largest].riskScore)
            largest = right;
        if (largest == i)
            break;
        swapEvent(&priorityQueue[i],
                &priorityQueue[largest]);
        i = largest;
    }
    return result;
}
int main() {
    AVLNode *root = NULL;
    Event events[] = {
        {"192.168.1.10",
         "10:00:01",
         "Login Attempt",
         30},
        {"10.0.0.15",
         "10:00:05",
         "Port Scan",
         90},
        {"192.168.1.10",
         "10:00:08",
         "Login Attempt",
         70},
```

```
        {"172.16.0.5",
         "10:00:12",
         "File Access",
         40},
        {"10.0.0.15",
         "10:00:15",
         "Port Scan",
         95},
        {"192.168.1.20",
         "10:00:20",
         "Access Request",
         50}
    };
    int n = sizeof(events) / sizeof(events[0]);
    clock_t start, end;
    start = clock();
    for (int i = 0; i < n; i++) {
        insertHash(events[i].ip);
        root = insertAVL(root, events[i]);
        insertPriority(events[i]);
    }
    end = clock();
    double executionTime =
        ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\n=====================================\n");
    printf(" CYBER THREAT DETECTION ENGINE\n");
    printf("=====================================\n");
    printf("\nTotal Events Processed: %d\n", n);
    printf("\nDuplicate IP Detection:\n");
    for (int i = 0; i < n; i++) {
        int count = searchHash(events[i].ip);
        if (count > 1) {
            printf("IP: %-15s Occurrences: %d\n",
                   events[i].ip, count);
        }
    }
    printf("\nEvents Ordered by Risk Score:\n");
    inorder(root);
    printf("\nHighest Risk Event:\n");
    Event highRisk = getHighestRisk();
    printf("IP Address : %s\n", highRisk.ip);
    printf("Event Type : %s\n", highRisk.eventType);
    printf("Risk Score : %d\n", highRisk.riskScore);
    printf("\nProcessing Time: %f seconds\n",
           executionTime);
    return 0;
}
```

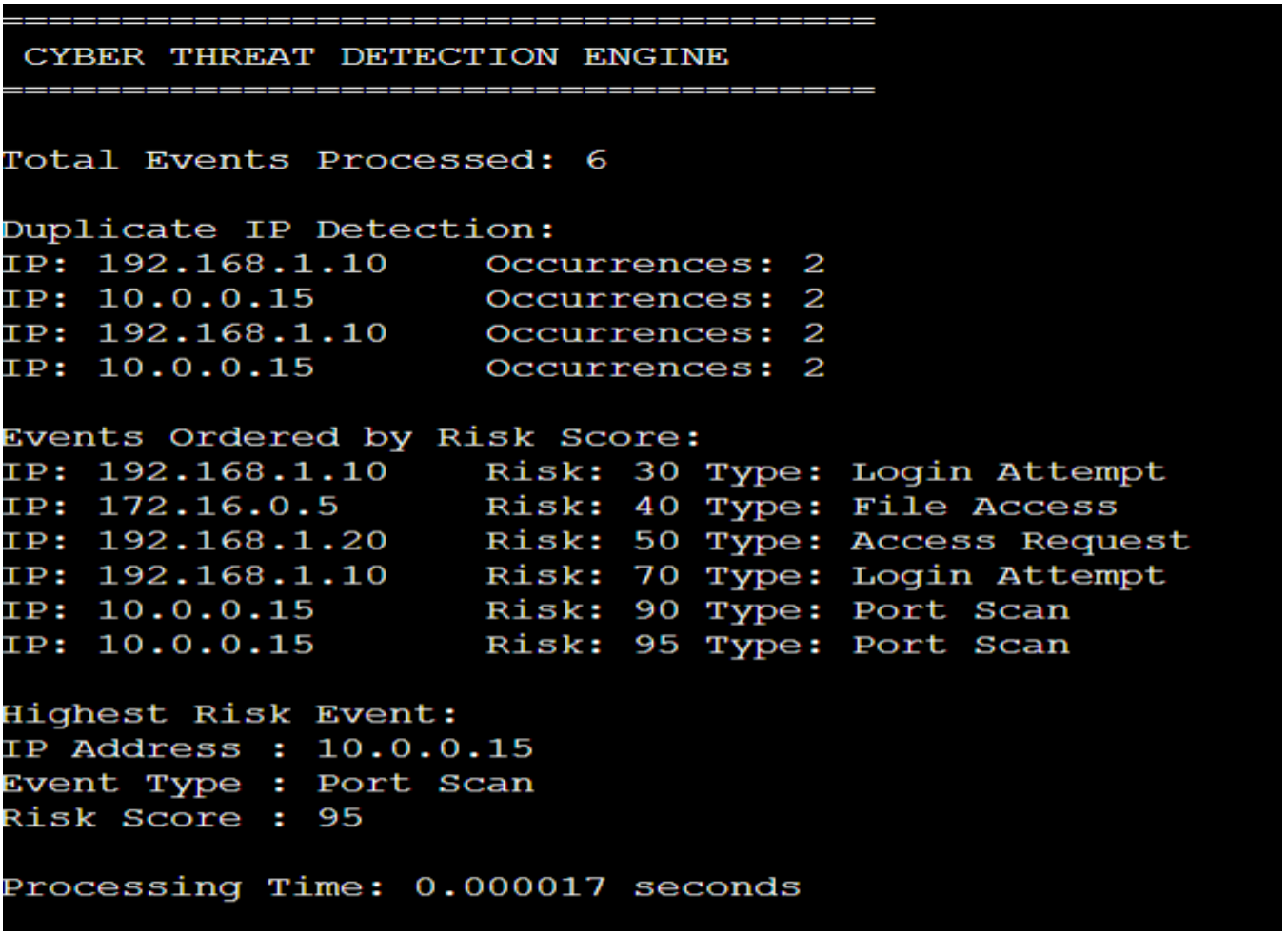

Figure 3: Output

4. Use of Modern Tools

The implementation can be developed using the following tools:

Tool	Purpose
C Programming Language	Implementation of data structures and algorithms
GCC / MinGW	Compilation and execution
Visual Studio Code / Code:Blocks	Program development
Git	Version control
GitHub	Source-code upload and project submission
Terminal / Command Prompt	Program execution and testing
CSV/Text Dataset	Storage of test security events

The program should be divided into modular components such as:

- Hash table module
- AVL Tree module
- Priority Queue module
- Event-generation module
- Performance-testing module
- Main program

Appropriate screenshots of program execution, test results, and GitHub repository should be included in the final submission.

5. Results and Validation

The system should be tested using different numbers of events and different hash-table load factors.

Suggested test sizes:

Test Case	Number of Events
Test 1	10,000
Test 2	50,000
Test 3	100,000
Test 4	500,000
Test 5	1,000,000

Suggested load factors:

Test	Load Factor
Low	0.25
Medium	0.50
High	0.75
Very High	0.90

The following measurements should be recorded:

- Hash insertion time
- Hash search time
- Number of collisions
- Average search time
- AVL insertion time
- Priority queue insertion time

Data Distribution Testing

At least two types of distributions should be considered.

Uniform Distribution

Events are distributed approximately evenly among different IP addresses.

Expected behavior:

- Fewer repeated IP addresses
- Relatively uniform hash-table usage
- Lower collision concentration

Skewed Distribution

A small number of IP addresses generate a large percentage of events.

Expected behavior:

- High number of duplicate IP addresses
- More frequent accesses to specific hash-table chains
- Higher possibility of concentrated collision activity
- Useful for simulating malicious IP behavior

Result Table Format

Load Factor	Distribution	Events	Collisions	Search Time	Insertion Time
0.25	Uniform	100,000	To be measured	To be measured	To be measured
0.50	Uniform	100,000	To be measured	To be measured	To be measured
0.75	Uniform	100,000	To be measured	To be measured	To be measured
0.90	Uniform	100,000	To be measured	To be measured	To be measured
0.25	Skewed	100,000	To be measured	To be measured	To be measured
0.50	Skewed	100,000	To be measured	To be measured	To be measured
0.75	Skewed	100,000	To be measured	To be measured	To be measured
0.90	Skewed	100,000	To be measured	To be measured	To be measured

The actual values should be obtained by running the C program. They should not be estimated or manually fabricated.

Validation

The implementation can be validated using the following test cases:

1. Insert a new IP address and verify successful insertion.
2. Search for an existing IP address.
3. Search for an IP address that does not exist.
4. Insert the same IP multiple times and verify duplicate detection.
5. Insert events with different risk scores and verify AVL ordering.
6. Insert multiple events into the priority queue and verify that the highest-risk event is returned first.
7. Test the system using a large number of events.
8. Compare performance at different load factors.
9. Compare uniform and skewed IP distributions.

6. Analysis and Engineering Decision

A hash table is selected for duplicate-IP detection because its average search and insertion complexity is O(1). This is particularly useful when millions of events must be processed.

An AVL Tree is selected for ordered risk-based storage because it maintains balance automatically and provides O(log n) search and insertion. Unlike the hash table, it also maintains the elements in an ordered structure, making it suitable for range queries and ordered reports.

A priority queue is selected because the system needs to continuously identify the highest-risk event. A max-heap provides O(1) access to the highest-priority event and O(log n) insertion and deletion.

The combination of these structures is more suitable than relying on a single data structure because each structure solves a different requirement.

Requirement	Suitable Data Structure	Reason
Duplicate IP detection	Hash Table	Average O(1) lookup
Collision handling	Chaining	Handles multiple keys at one index
Ordered risk storage	AVL Tree	O(log n) balanced operations
Range queries	AVL Tree	Maintains sorted order
Highest-risk event	Priority Queue	Fast highest-priority access
Large-scale processing	Combination	Each structure handles a specific task

Therefore, empirical testing is necessary to determine how well the selected hashing strategy performs under different conditions.

7. Broader Considerations

Sustainability

Efficient algorithms reduce unnecessary computation and help security systems process large volumes of data with limited resources. Selecting suitable data structures can improve processing efficiency and reduce computational overhead.

Society

Cyber threat detection systems help protect digital infrastructure, organizations, and users from malicious activities. Rapid identification of suspicious events can reduce the impact of security incidents.

Ethics

Security monitoring systems must be designed responsibly. Event data, particularly IP addresses and access information, should be handled securely and only used for legitimate security purposes.

Professional Responsibility

The implementation should be tested carefully because incorrect threat detection can result in either missed attacks or unnecessary alerts. Performance measurements should represent actual experimental results rather than assumed values.

SDG 9

The project is related to SDG 9 because it applies computing and cybersecurity technologies to improve digital infrastructure, innovation, and reliable information systems.

SDG 16

The project supports SDG 16 by contributing to secure and resilient digital infrastructure and helping organizations detect potentially harmful activities.

8. Conclusion

The proposed Real-Time Cyber Threat Detection Engine combines a hash table, AVL Tree, and priority queue to efficiently process large volumes of security events.

The hash table provides fast average-case duplicate-IP detection, while the AVL Tree maintains events in a dynamically balanced, risk-ordered structure. The priority queue allows the system to continuously identify the highest-risk events.

The project also evaluates hash-table performance under different load factors and data distributions. This analysis helps demonstrate how collision frequency and data distribution influence practical performance.

The proposed solution satisfies the major requirements of high-volume processing, duplicate detection, risk ranking, ordered reporting, and performance evaluation. Further improvements could include parallel event processing, more advanced threat-detection rules, dynamic hash-table resizing, and integration with real-time security monitoring platforms.

9. Student Reflection

This assignment helped in understanding how different data structures can be combined to solve a real-world computing problem rather than using one data structure for every operation.

The project provided practical experience in implementing hashing, collision resolution, AVL Tree balancing, and priority queues in C. It also demonstrated the difference between theoretical time complexity and practical performance under different data distributions.

One important learning was that the performance of a hash table depends not only on the number of elements but also on the load factor and distribution of keys. Testing skewed data was particularly useful because real-world cybersecurity data may contain repeated activity from a small number of suspicious IP addresses.

The system could also be extended to support real-time network input and visualization of detected threats.

10. References

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, MIT Press.
2. Ellis Horowitz, Sartaj Sahni, and Susan Anderson-Freed, *Fundamentals of Data Structures in C*, W. H. Freeman.
3. Robert Lafore, *Data Structures & Algorithms in C++*, Pearson Education.
4. Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser, *Data Structures and Algorithms in Java*, Wiley.
5. William Stallings, *Computer Security: Principles and Practice*, Pearson.
6. Behrouz A. Forouzan, *Cryptography and Network Security*, McGraw-Hill Education.
7. William Stallings, *Network Security Essentials: Applications and Standards*, Pearson.
8. National Institute of Standards and Technology (NIST), *Computer Security Incident Handling Guide*, NIST Special Publication 800-61.
9. National Institute of Standards and Technology (NIST), *Cybersecurity Framework (CSF)*, NIST.
10. OWASP Foundation, *OWASP Application Security Verification Standard (ASVS)* and cybersecuri

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	**100**

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation			L3/L4	10
Application of knowledge			L3/L4	20
Solution / Design / Implementation			L4/L5/L6	20
Modern tool usage			L3/L4	10
Validation			L4/L5	15
Analysis & justification			L4/L5	15
Broader considerations			L4/L5	5
Documentation & reflection			L3/L4	5

Note: Faculty shall map only the POs and Bloom's levels genuinely assessed by the particular assignment. Every assignment is **not required to map to every PO**.

H. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	≥ 70%
Level 2	60–69%
Level 1	50–59%
Level 0	< 50%

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action:

Action to be implemented in the next offering: